

OpenClaw Target Tracking

OpenClaw Target Tracking

1. Course Content
 - Course Overview
 - Learning Objectives
2. Preparation
3. Target Tracking Case
4. Source Code Analysis
 - 4.1 `TargetTrack()` — Entry Point for Starting/Canceling Tracking
 - 4.2 `_gaze_track_loop()` — Background Tracking Thread Loop
 - 4.3 `_get_bbox()` — Retrieving the Target Bounding Box Center
 - 4.4 `_gaze_control()` — Gaze Control (Single-Joint Adjustment)
5. Parameter Tuning
6. Common Issues and Solutions

1. Course Content

Course Overview

- OpenClaw target tracking is the core function of robot vision servo control. It tracks target objects in real time using a camera and controls the robotic arm joints to adjust the camera angle, keeping the target centered in the frame.
- The target tracking function relies on the preceding GetBbox (target detection) to obtain the initial ROI bounding box, and then starts the tracking thread to continuously correct the robotic arm joint angles.
- The tracking function supports both starting and canceling tracking. It automatically pauses when the target confidence is too low or the target is lost, and automatically resumes tracking when the target reappears.

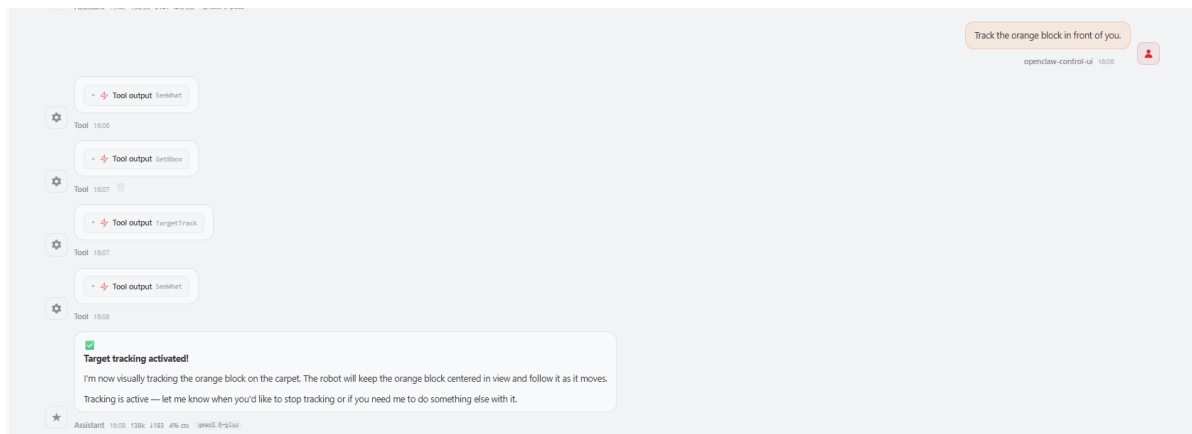
Learning Objectives

1. Master the methods for starting and canceling target tracking using OpenClaw.

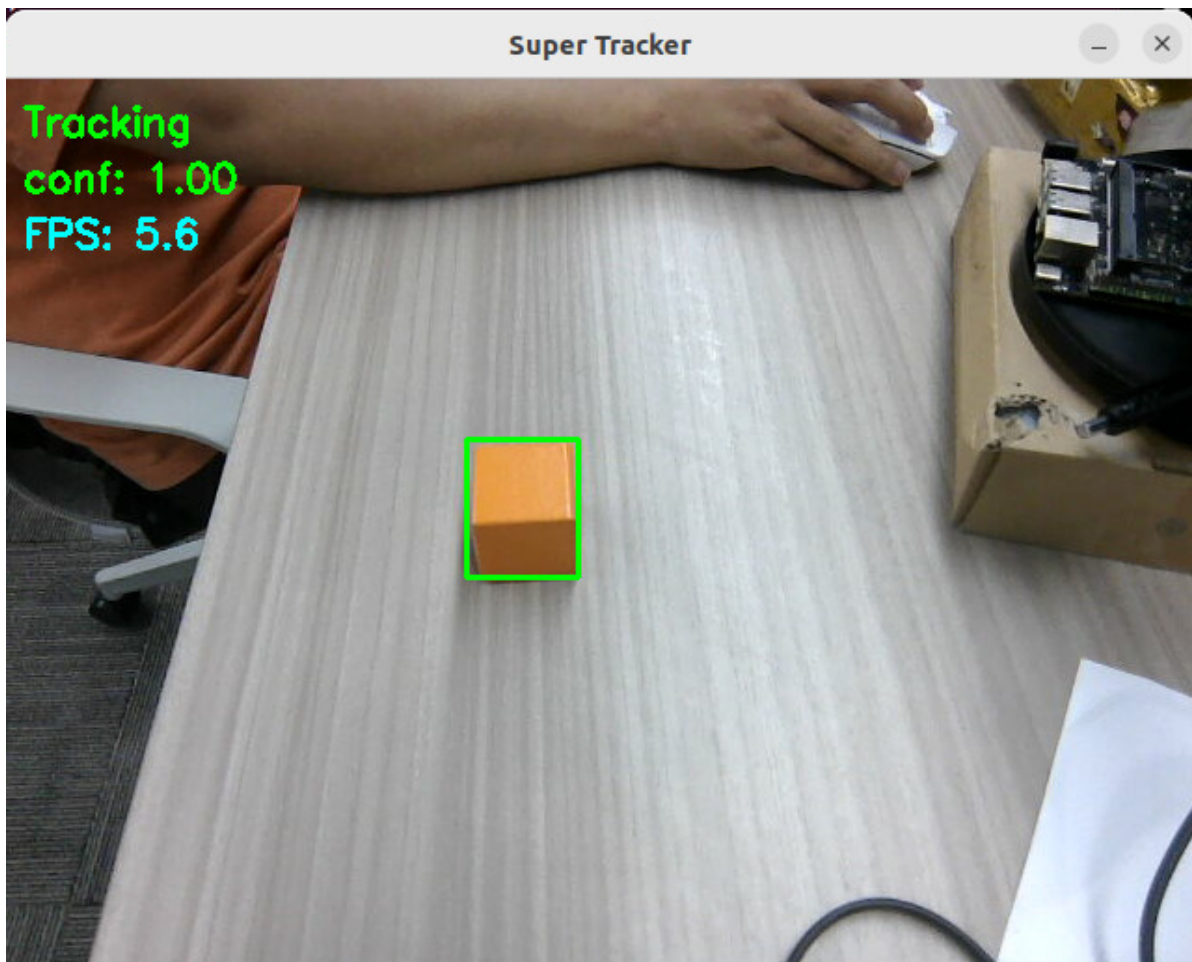
2. Preparation

- Start the MicroROS chassis agent (if it's already started, no need to start it again)

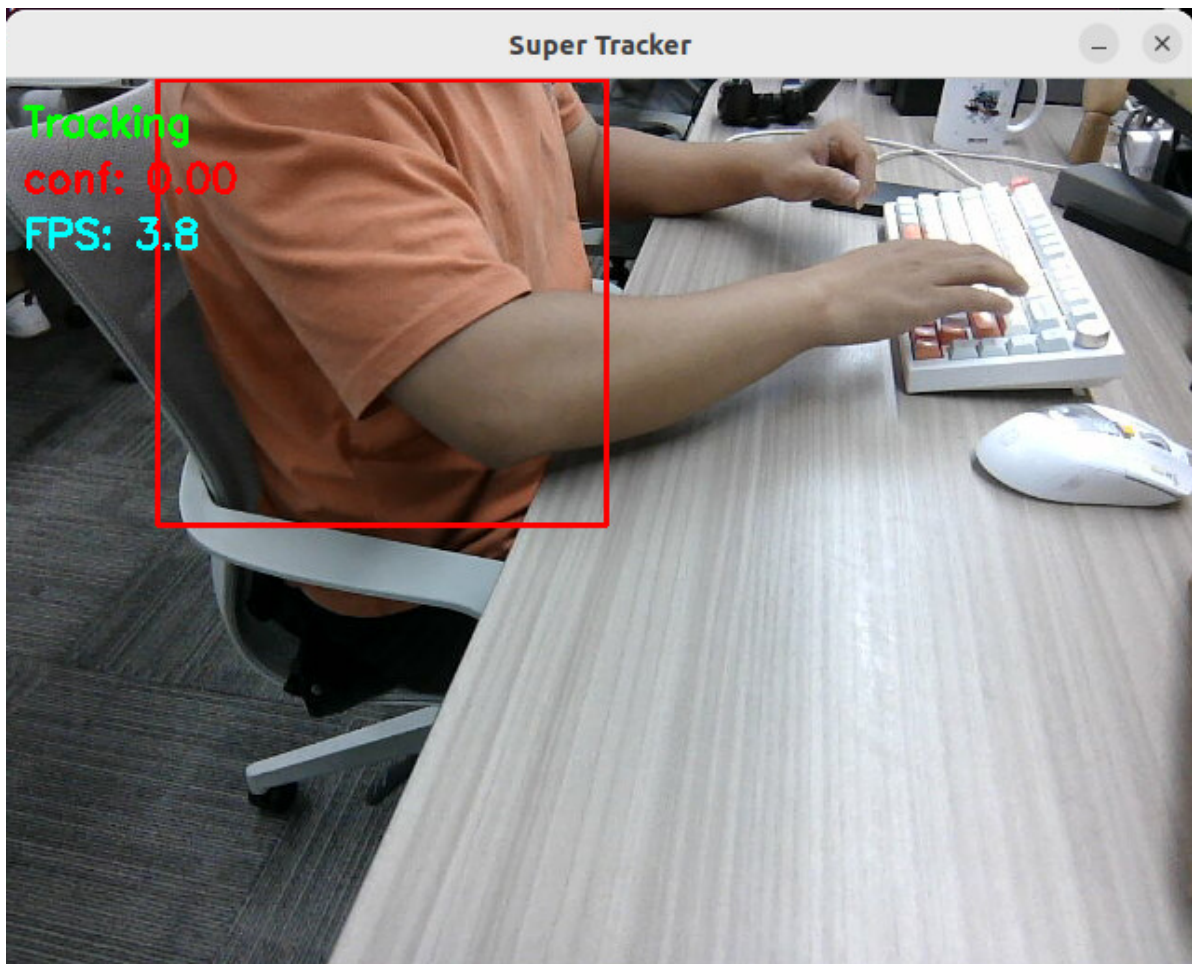
```
sh start_agent.sh
```

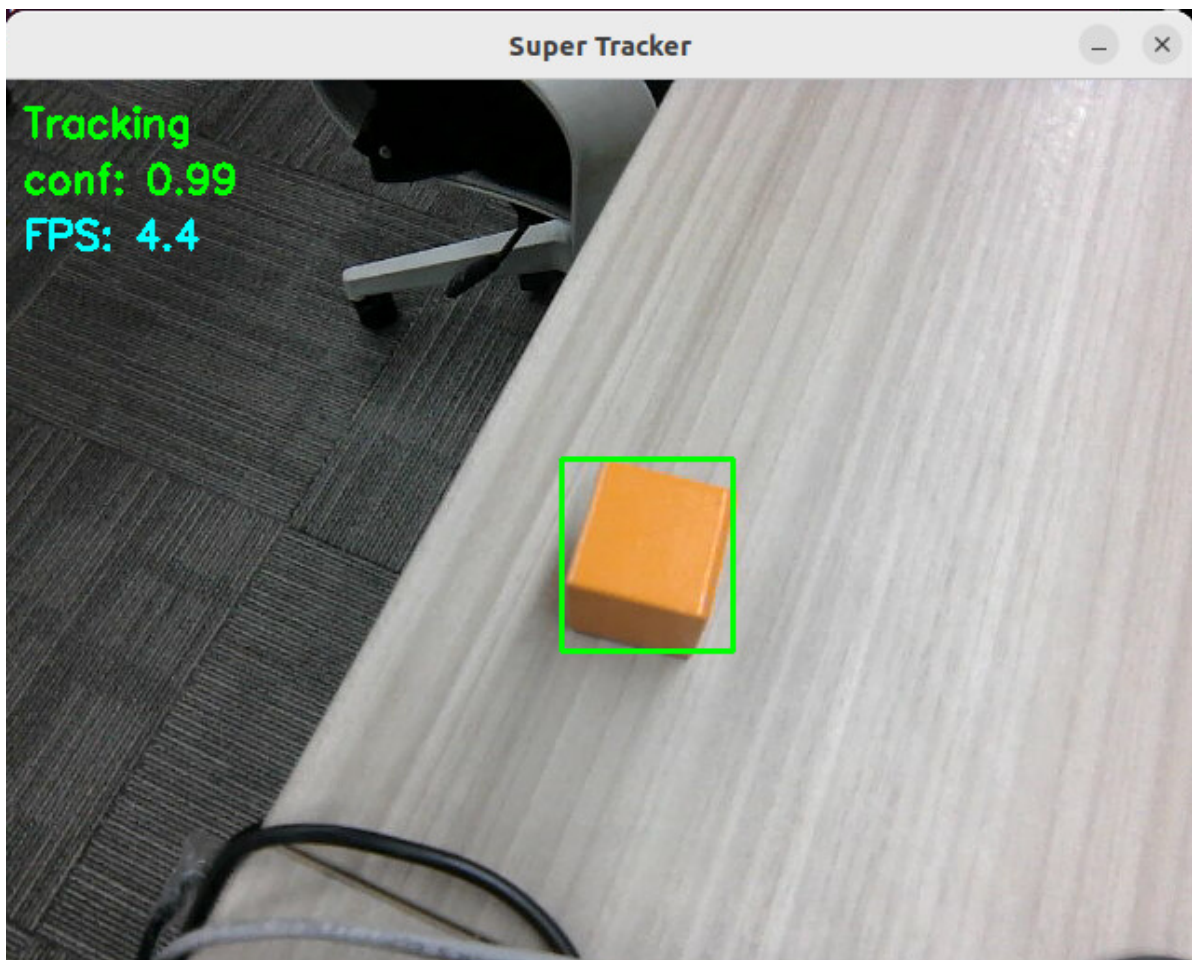
- As the target object moves, the robotic arm adjusts its joint angles to ensure the camera remains continuously focused on the object.



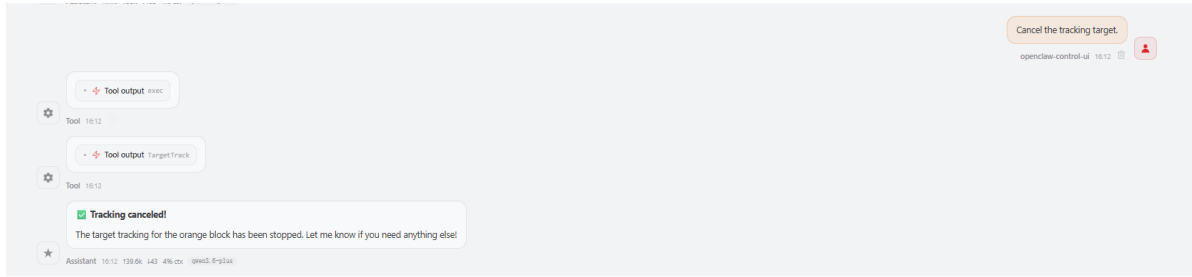
- If the target's confidence score drops too low or the target is lost from the field of view, the robotic arm will cease movement.



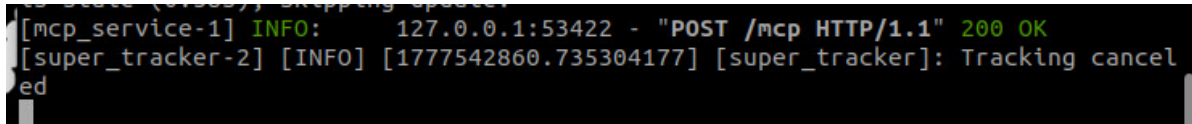
- When the target reappears within the field of view, the tracker automatically searches for it; once the confidence score exceeds 0.1, the robotic arm resumes tracking.



- To stop tracking the target, you can issue a new command to OpenClaw.



- The MCP server logs a message indicating that tracking of the target has been terminated.



[!IMPORTANT]

- In the [02-OpenClaw Robotic Arm Tracking and Grasping] module, OpenClaw must first successfully track the target before it can proceed to grasp it.

4. Source Code Analysis

- Source Code Path

```
$HOME/M3Pro_ws/src/multi_brains_pre/multi_brains_pre/action_controller.py
```

The following section provides an explanation of the source code for the core target tracking functions within the `ActionController` class—specifically `TargetTrack()` and the internal functions it invokes.

4.1 `TargetTrack()` — Entry Point for Starting/Canceling Tracking

```
def TargetTrack(self, x1_or_cmd, y1:int=None, x2:int=None, y2:int=None):
    # Cancel tracking mode
    if isinstance(x1_or_cmd, str):
        if x1_or_cmd != "cancel":
            return [False, "Invalid command for TargetTrack. Use 'cancel' to stop tracking."]
        # Stop gaze thread first
        if hasattr(self, '_gaze_stop_event'):
            self._gaze_stop_event.set()
        if hasattr(self, '_gaze_thread') and self._gaze_thread is not None and self._gaze_thread.is_alive():
            self._gaze_thread.join(timeout=2.0)
            self._gaze_thread = None
        # Cancel tracker service
        if not self.cancel_track_client.wait_for_service(timeout_sec=5.0):
            return [False, "CancelTrack service not available"]
        self.cancel_track_client.call(CancelTrack.Request())
        # Reset arm to initial pose
        self._reset_joints()
        self.InitArmPose()
        return [True, "Tracking cancelled and arm reset"]
```

```

# Start tracking mode
x1: int = x1_or_cmd
if not self.start_track_client.wait_for_service(timeout_sec=5.0):
    return [False, "Tracking service not available"]
result = self.start_track_client.call(StartTrack.Request(x1=x1, y1=y1,
x2=x2, y2=y2))
if not result.success:
    return [False, "Tracking failed"]
# Start gaze control thread
self._gaze_stop_event = threading.Event()
self._gaze_thread = threading.Thread(target=self._gaze_track_loop,
daemon=True)
self._gaze_thread.start()
return [True, "Tracking started"]

```

Code Explanation:

1. **Cancel Tracking:** If the input parameter is the string "cancel", the following steps are executed sequentially: stop the gaze-tracking thread → cancel the tracking service → restore the robotic arm to its initial pose.
2. **Start Tracking:** Calls `start_track_client` (a ROS2 Service Client) to initiate the target tracking service, passing the target ROI bounding box coordinates `(x1, y1, x2, y2)`.
3. **Background Thread:** Launches a separate thread, `_gaze_track_loop`, to continuously execute visual servo control without interfering with the main thread's ability to respond to other commands.
4. **Parameter Validation:** The "start" mode requires a complete set of coordinate parameters; if any coordinate is missing, an error is returned.

4.2 `_gaze_track_loop()` — Background Tracking Thread Loop

```

def _gaze_track_loop(self):
    '''Background thread: continuously call _gaze_control to follow the tracked target.'''
    self._get_joints()
    while not self._gaze_stop_event.is_set() and rclpy.ok():
        box = self._get_bbox()
        if box[2] < 0.08:
            # confidence too low, wait for next frame
            time.sleep(0.1)
            continue
        error_x = box[0] - 320
        error_y = box[1] - 240
        self._gaze_control(error_x, error_y)
        time.sleep(0.25)

```

Code Explanation:

1. **Joint Initialization:** Calls `_get_joints()` to retrieve the current angle values for robot arm joints 1, 2, and 4.
2. **Loop Conditions:** Uses `_gaze_stop_event` and `rclpy.ok()` as termination conditions to ensure the thread can be safely stopped.

3. **Confidence Filtering:** If the target confidence score (`box[2]`) falls below 0.08, the current loop iteration is skipped to wait for the next frame; this corresponds to the behavior described in the documentation as "the robot arm stops moving when the target is lost."
 4. **Error Calculation:** `error_x = center_x - 320` and `error_y = center_y - 240` calculate the pixel deviation using the image center (320, 240) as the reference point.
 5. **Gaze Control:** Calls `_gaze_control()` to adjust the joint angles based on the calculated errors, thereby centering the target within the frame.
-

4.3 `_get_bbox()` — Retrieving the Target Bounding Box Center

```
def _get_bbox(self)-> list:
    with self._track_lock:
        center_x=(self.track_box.bbox[0]+self.track_box.bbox[2])/2
        center_y=(self.track_box.bbox[1]+self.track_box.bbox[3])/2
        msg_stamp_sec = self.track_box.timestamp.sec +
self.track_box.timestamp.nanosec * 1e-9
        now_sec = self.get_clock().now().nanoseconds * 1e-9
        confidence = self.track_box.confidence if (now_sec - msg_stamp_sec) <
0.5 else 0.0
        return [center_x, center_y, confidence]
```

Code Explanation:

1. **Bounding Box Center Calculation:** `(bbox[0]+bbox[2])/2` calculates the center along the X-axis, while `(bbox[1]+bbox[3])/2` calculates the center along the Y-axis.
 2. **Thread Safety:** Access to the `track_box` data is protected by the `_track_lock` mutex.
 3. **Confidence Timeliness:** If a message timestamp exceeds the current system time by more than 0.5 seconds, the confidence score is forcibly set to 0 to prevent the use of stale tracking data.
-

4.4 `_gaze_control()` — Gaze Control (Single-Joint Adjustment)

```
def _gaze_control(self, error_x, error_y):
    '''Eye gaze follows the target object'''
    if self.joint_1 is None or self.joint_2 is None or self.joint_4 is None:
        return

    if abs(error_x) > self.gaze_threshold:
        if error_x > self.gaze_threshold:
            self.joint_1 += self.step
        elif error_x < -self.gaze_threshold:
            self.joint_1 -= self.step
        self.SingleJoint_pub.publish(ArmJoint(id=1, joint=self.joint_1,
time=100))

    if abs(error_y) > self.gaze_threshold:
        if error_y > self.gaze_threshold:
            self.joint_4 -= self.step
        elif error_y < -self.gaze_threshold:
```

```

        self.joint_4 += self.step
        # Limit joint_4 to prevent damage
        if self.joint_4 > -5:
            self.SingleJoint_pub.publish(ArmJoint(id=4, joint=self.joint_4,
time=100))
        else:
            # joint_4 at lower limit, adjust joint_2 to continue gaze control
            self.joint_2 -= self.step
            self.SingleJoint_pub.publish(ArmJoint(id=2, joint=self.joint_2,
time=100))

```

Code Explanation:

1. **X-Direction Control (Joint 1):** When the error exceeds `gaze_threshold`, Joint 1 (horizontal rotation) is used to adjust the camera's left-right field of view.
2. **Y-Direction Control (Joint 4):** When the error exceeds the threshold, Joint 4 (pitch) is used to adjust the camera's up-down field of view.
3. **Joint Limit Protection:** When Joint 4 reaches its lower limit ($< -5^\circ$), the system automatically switches to Joint 2 for compensation to prevent damage to the robotic arm.
4. **Single-Joint Publishing:** Single-joint control commands are published via the `SingleJoint_pub` topic, adjusting the joint by `step` degrees with each command.

5. Parameter Tuning

- Parameter file path:

```
$HOME/M3Pro_ws/src/m3pro_bringup/config/common.yaml
```

- Parameters related to the tracking function:

```

...
#gaze control
gaze_threshold: 55.0          # Error threshold for gaze control (units:
pixels)
gaze_step: 2                  # Step size for gaze control (units: joint
degrees)
...

```

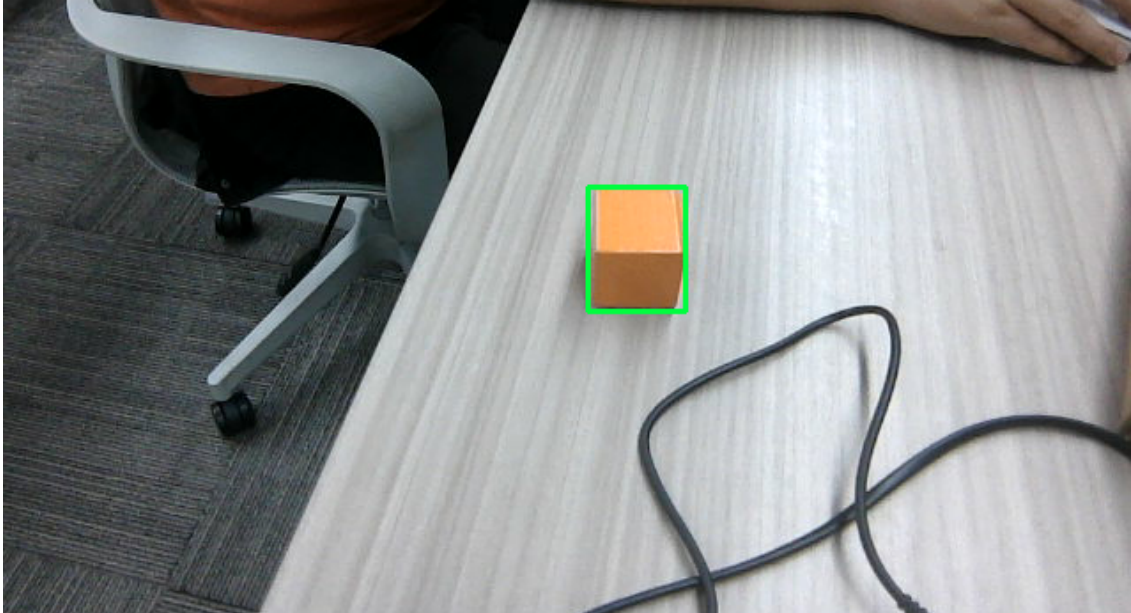
Parameter	Effect of Increasing Value	Effect of Decreasing Value
<code>gaze_threshold</code>	Allows for larger deviations; adjustments are more lenient; no adjustments are made if the deviation is below this value.	Requires more precise alignment; adjustments occur more frequently; setting this value too low may cause the robotic arm to exhibit steady-state oscillations.
<code>gaze_step</code>	Increases the adjustment step size.	Smooths the adjustment process, but increases the risk of overshoot.

6. Common Issues and Solutions

- **Inaccurate Target ROI Box During Tracking**
- Check whether the ROI box—generated by the `GetBbox` function during the initial selection of the target object—is accurate. For detailed steps, refer to: [03-openclaw Development] — [05-openclaw Multimodal Vision] — [4.4 Obtaining Target Bounding Box Coordinates].

- The verification image can be found at:

`$HOME/M3Pro_ws/multi_brains_file/verify_image.png`



- If the visual model's bounding box selections are inaccurate, you need to change the visual model within Dify. Log in to the Dify Admin Console and update the visual model configured for "Robot Vision."

